gemstone-rs Companion Manual

Setup, user manual, examples, cookbook, codegen, explorer, and workbench in one PDF

Generated from the Markdown sources in gemstone-rs/docs.

Contents

gemstone-rs Documentation

This directory contains the human-facing guides for `gemstone-rs`.

Start Here

Guide	Use it when
Setup Guide	You need Rust, GemStone environment variables, install commands, and first login checks.
Examples Guide	You want to know which example to run for each feature.
User Manual	You want the main Rust API surface in one place.
Cookbook	You want task-focused recipes for sessions, transactions, browser calls, codegen, explorer, and VS Code.
gemstone-py vs gemstone-rs	You want install paths, use cases, maturity, and feature differences between the Python and Rust projects.
BridgeRoot and Object Mapping	You want MagLev-style bridge-root storage and explicit Rust-to-GemStone value mapping.
Codegen Guide	You want generated Rust wrappers for GemStone classes and methods.
Explorer Guide	You want the local HTTP explorer API and safety defaults.
VS Code Workbench	You want sidebar browsing, codegen preview/diff/generate, and explorer launch commands.
Medium Article	You want an article-style explanation suitable for publishing or sharing.
Funny Introduction	You want a lighter multi-part tour of the same concepts.
Release Checklist	You want the crate, VSIX, GitHub Release, and post-release verification flow.

PDFs

Generated PDFs live in pdf/. Rebuild them after Markdown changes:

```
python3 docs/build_pdf_docs.py
```

Shortest Path

```
cargo install gemstone-rs-cli
gemstone-rs --help
```

For library use:

```
cargo add gemstone-rs
```

For local examples from a checkout:

```
cd /path/to/gemstone-rs
cargo run -p gemstone-rs --example quickstart
```

For the local explorer:

```
cargo install gemstone-rs-explorer
gemstone-rs-explorer --port 8787
```

For VS Code:

```
https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-rs-workbench
```

Setup Guide

This guide gets you from a Rust project or source checkout to a live GemStone login.

What You Need

At minimum:

- Rust stable toolchain
- access to a GemStone/S 64 stone
- GemStone client library access through `libgcirpc`
- GemStone username and password

For the full local development experience:

- a `gemstone-rs` checkout
- `cargo`, `rustfmt`, and `clippy`
- Node.js 22 when working on the VS Code extension
- a Marketplace PAT only when publishing the VSIX
- a crates.io API token only when publishing crates

Which Install Path Should I Use?

Use case	Command
Rust application dependency	`cargo add gemstone-rs`
CLI tools	`cargo install gemstone-rs-cli`
Local HTTP explorer	`cargo install gemstone-rs-explorer`
Source checkout examples	`cargo run -p gemstone-rs --example quickstart`
VS Code users	Install `unicompute.gemstone-rs-workbench` from Marketplace

Environment Variables

Most live commands use `Config::from_env()`:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

`GS_STONE` is canonical. `GS_STONE_NAME` is accepted as an alias when `GS_STONE` is absent. Setting both to the same value keeps CLI, examples, explorer, and VS Code settings aligned.

Optional variables:

```
export GS_HOST=localhost
export GS_NETLDI=netldi
export GS_GEM_SERVICE=gemnetobject
export GS_HOST_USERNAME=
export GS_HOST_PASSWORD=
export GS_LIB_PATH=/full/path/to/libgcirpc.dylib
```

Variable	Required	Purpose
`GS_LIB`	Usually	GemStone `lib/` directory for runtime library discovery.
`GS_LIB_PATH`	No	Full path to a specific `libgcirpc` file.
`GS_STONE`	Yes	Stone name.
`GS_STONE_NAME`	No	Stone-name alias used when `GS_STONE` is absent.
`GS_USERNAME`	Yes	GemStone login username.
`GS_PASSWORD`	Yes	GemStone login password.
`GS_HOST`	No	Remote host, defaults to local behavior.
`GS_NETLDI`	No	NetLDI service name.
`GS_GEM_SERVICE`	No	Gem service name.

First Login

With the CLI:

```
cargo install gemstone-rs-cli
gemstone-rs eval "3 + 4"
```

Expected output:

```
7
```

From a source checkout:

```
cd /path/to/gemstone-rs
cargo run -p gemstone-rs --example hello_gemstone
cargo run -p gemstone-rs --example quickstart
```

From a Rust application:

```
use gemstone_rs::{Config, Session, Value};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;
    assert_eq!(session.eval("3 + 4")?, Value::SmallInt(7));
```

```
    Ok(())
}
```

Development Checkout

```
git clone git@github.com:unicompute/gemstone-rs.git
cd gemstone-rs
make verify
```

`make verify` runs formatting, `cargo check`, clippy, tests, codegen freshness, and the VS Code extension syntax check.

Package the VSIX locally:

```
make vscode-package
```

Verify already-published artifacts:

```
scripts/publish_verify.sh 0.2.0
```

User Manual

`gemstone-rs` is the safe Rust client API for GemStone/S over GCI. Rust code imports it as `gemstone_rs`.

Configuration

Use `Config::from_env()` for normal applications:

```
let config = gemstone_rs::Config::from_env()?;
```

Use the builder when you want explicit configuration:

```
use gemstone_rs::Config;

let config = Config::builder()
    .stone("gs64stone")
    .host("localhost")
    .netldi("netldi")
```

```
    .username("DataCurator")
    .password("swordfish")
    .build()?;
```

Sessions

`Session` logs out automatically on drop. Calling `logout()` explicitly is still useful in examples and command-line tools.

```
use gemstone_rs::{Config, Session, Value};

let mut session = Session::login(Config::from_env()?)?;
let value = session.eval("3 + 4")?;
assert_eq!(value, Value::SmallInt(7));
session.logout()?;
```

`Session` is deliberately not `Send` or `Sync`. Keep a session on the thread that logged it in.

Eval and Perform

`eval` returns a marshalled `Value`:

```
let value = session.eval("3 + 4")?;
```

`execute` returns a raw `Oop`:

```
let object_class = session.execute("Object")?;
```

`perform` returns a marshalled `Value`:

```
let seven = session.smallint_oop(7);
let printed = session.perform(seven, "printString", &[])?;
```

`perform_oop` returns a raw `Oop`:

```
let printed_oop = session.perform_oop(seven, "printString", &[])?;
let printed = session.fetch_string(printed_oop)?;
```

OOP and Value Conversion

`Value` covers nil, booleans, small integers, characters, fetched strings, and raw OOPs:

```
use gemstone_rs::Value;

let oop = session.value_to_oop(&Value::SmallInt(7))?;
let value = session.eval("true")?;
println!("{value:?}");
```

Helpers:

```
let nil = session.nil_oop();
let yes = session.bool_oop(true);
let seven = session.smallint_oop(7);
let text = session.new_string("hello")?;
let symbol = session.new_symbol("ExampleSymbol")?;
```

Globals

`global_get` and `global_put` use `UserGlobals`:

```
let text = session.new_string("hello from Rust")?;
session.global_put("GemStoneRsText", text)?;
let stored = session.global_get("GemStoneRsText")?;
println!("{}", session.fetch_string(stored)?);
```

Transactions

Use `transaction` when you want commit-on-success and abort-on-error:

```
session.transaction(|session| {
    let value = session.new_string("committed")?;
    session.global_put("GemStoneRsCommitted", value)
})?;
```

Manual transaction primitives are also available:

```
let needs_commit = session.needs_commit()?;
let in_transaction = session.in_transaction()?;
```

```
session.commit()?;
session.abort()?;
```

Export-Set Lifetime

Use `retain_oop` when a raw OOP must be held across calls and protected from GemStone-side collection:

```
let text = session.new_string("retained")?;
let handle = session.retain_oop(text)?;
println!("{}", handle.oop().raw());
handle.release()?;
```

The handle releases on drop if you do not call `release()`.

Browser API

```
use gemstone_rs::browser::{Browser, ALL_PROTOCOLS};

let mut browser = Browser::new(&mut session);
let dictionaries = browser.dictionaries()?;
let classes = browser.classes("UserGlobals")?;
let protocols = browser.protocols("Object", false, "")?;
let methods = browser.methods("Object", ALL_PROTOCOLS, false, "")?;
let source = browser.source("Object", "printString", false, "")?;
```

Pass an empty dictionary to resolve through the active user's symbol list.

Error Handling

Most APIs return `gemstone_rs::Result<T>`. Errors distinguish missing environment, missing config, GCI loading/calls, GemStone errors, illegal OOPs, unexpected typed codegen returns, and conversion issues.

```
match Config::from_env() {
    Ok(config) => println!("stone {}", config.stone),
    Err(err) => eprintln!("configuration error: {err}"),
}
```

Examples Guide

The `gemstone-rs` examples are split between runnable Cargo examples and the checked-in codegen sample under the repository-level `examples/` directory.

Common Setup

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

Run Cargo examples from the repository root:

```
cd /path/to/gemstone-rs
cargo run -p gemstone-rs --example quickstart
```

Example Map

Feature	Command or path	What it demonstrates
First login	`cargo run -p gemstone-rs --example hello_gemstone`	Reads env config, logs in, prints a session id, and evaluates `3 + 4`.
Quickstart	`cargo run -p gemstone-rs --example quickstart`	Eval, `global_put`, `global_get`, string fetch, cleanup.
Eval only	`cargo run -p gemstone-rs --example eval`	Minimal `Session::eval` shape.
Browser API	`cargo run -p gemstone-rs --example browser`	Dictionaries, protocols, methods, and source.
Transactions	`cargo run -p gemstone-rs --example transactions`	Commit-on-success and abort-on-error transaction wrapper.
OOP/value conversion	`cargo run -p gemstone-rs --example oop_values`	`Value`, `Oop`, strings, symbols, and export-set retention.

Feature	Command or path	What it demonstrates
BridgeRoot mapping	`cargo run -p gemstone-rs --example bridge_root_mapping`	MagLev-style bridge-root storage with explicit `BridgeValue` mapping.
Derive mapping	`cargo run -p gemstone-rs --example derive_mapping`	`#[derive(BridgeMapped)]`, symbol keys, nested structs, vectors, and BridgeRoot transactions.
Offline codegen	`cargo run -p gemstone-rs --example codegen_preview`	Generates wrappers from the sample config without a live stone.
Codegen workflow	`cargo run -p gemstone-rs --example codegen_workflow`	Writes config, previews, diffs, checks, generates, and verifies a clean diff.
Codegen discovery	`cargo run -p gemstone-rs --example codegen_discover`	Connects to a live stone and discovers a starter config for `Object`.
Mapping discovery	`cargo run -p gemstone-rs --example codegen_discover_mapping`	Connects to a live stone and proposes a `BridgeMapped` config.
Generated wrapper app	`cargo run -p gemstone-rs --example generated_wrapper_app`	Uses checked-in generated wrappers to call `Object>>printString`.
Generated mapping app	`cargo run -p gemstone-rs --example generated_mapping_app`	Uses codegen-created `BridgeMapped` structs with `BridgeRoot`.
Codegen files	`examples/codegen/`	Config, generated wrappers, check/diff/generate workflow.
Explorer tooling	`examples/tooling/explorer.md`	Local explorer startup and endpoint checks.
VS Code tooling	`examples/tooling/vscode-workbench.md`	Sidebar browsing, codegen actions, and explorer launch.

Suggested Learning Order

1. `hello_gemstone`
2. `quickstart`
3. `browser`
4. `oop_values`
5. `transactions`
6. `bridge_root_mapping`
7. `derive_mapping`

8. `codegen_preview`
9. `codegen_workflow`
10. `generated_wrapper_app`
11. `generated_mapping_app`
12. `codegen_discover`
13. `codegen_discover_mapping`
14. `examples/codegen/`
15. `examples/tooling/explorer.md`
16. `examples/tooling/vscode-workbench.md`

Expected Output

Live examples need GemStone credentials and a reachable stone. If the environment is configured correctly, these are the important lines to look for:

```
$ cargo run -p gemstone-rs --example hello_gemstone
session id: <number>
3 + 4 => SmallInt(7)

$ cargo run -p gemstone-rs --example quickstart
GemStone eval ok: SmallInt(7)
GemStoneRsQuickstart: hello from gemstone-rs quickstart

$ cargo run -p gemstone-rs --example generated_wrapper_app
generated wrapper printString: 7

$ cargo run -p gemstone-rs --example bridge_root_mapping
bridge root: GemStoneRsBridgeRoot
MyTestDict OOP: <number>
loaded payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP" }

$ cargo run -p gemstone-rs --example derive_mapping
derived mapped payload: BookingDraft { amount: 100, customer: CustomerDraft { name:
"Tariq" }, tags: ["priority", "demo"] }

$ cargo run -p gemstone-rs --example generated_mapping_app
generated mapped payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP",
tags: ["priority", "demo"] }
```

Offline examples should run without GemStone:

```
$ cargo run -p gemstone-rs --example codegen_workflow
before generate: exists=false up_to_date=false diff_bytes=<number>
after generate: exists=true up_to_date=true
diff after generate: clean
```

CLI Equivalents

The CLI gives you the same live checks without compiling examples:

```
cargo install gemstone-rs-cli
gemstone-rs eval "3 + 4"
gemstone-rs browse dictionaries
gemstone-rs browse classes UserGlobals
gemstone-rs browse protocols Object
gemstone-rs browse methods Object "-- all --"
gemstone-rs browse source Object printString
gemstone-rs inspect oop 20
```

Codegen Workflow

```
cargo run -p gemstone-rs-cli -- codegen init examples/codegen/demo.codegen
cargo run -p gemstone-rs-cli -- codegen preview examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen diff examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen check examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen generate examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen discover-mapping examples/codegen/
mapping.codegen BookingDraft Object
```

Generate a starter config from a live stone:

```
cargo run -p gemstone-rs-cli -- codegen discover examples/codegen/discovered.codegen
Object
cargo run -p gemstone-rs --example codegen_discover_mapping
```

Run the generated wrapper example after checking the generated file into the repository:

```
cargo run -p gemstone-rs --example generated_wrapper_app
```

VS Code

Install the workbench from:

```
https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-rs-workbench
```

Use the GemStone RS sidebar to browse dictionaries, classes, protocols, and methods. The same sidebar exposes Codegen Discover, Preview, Diff, Check, Generate, Generate Mapping Config, Preview BridgeRoot, Run Generated Mapping Example, and Open Docs actions.

Later Examples

These are useful, but should wait until the corresponding APIs are stable:

- a tiny Axum or Actix web service using `gemstone-rs`
- a read-only class browser CLI walkthrough with captured output
- a local explorer workflow with screenshots
- a live smoke cookbook for CI secrets

Cookbook

This cookbook is a collection of direct `gemstone-rs` recipes. Each recipe is small enough to copy into a Rust CLI, service, or maintenance tool.

Recipe 1: Open a Session and Evaluate Smalltalk

```
use gemstone_rs::{Config, Session};

let mut session = Session::login(Config::from_env()?)?;
println!("{:?}", session.eval("3 factorial")?);
```

Use this when you need the smallest live sanity check.

Recipe 2: Verify `3 + 4`

```
use gemstone_rs::{Config, Session, Value};

let mut session = Session::login(Config::from_env()?)?;
assert_eq!(session.eval("3 + 4")?, Value::SmallInt(7));
```

This is the test to keep in every live smoke lane.

Recipe 3: Write and Read `UserGlobals`

```
use gemstone_rs::{Config, Oop, Session};

let mut session = Session::login(Config::from_env()?)?;
let key = "GemStoneRsCookbook";
let value = session.new_string("stored from Rust")?;
```

```
session.global_put(key, value)?;

let stored = session.global_get(key)?;
println!("{}", session.fetch_string(stored)?);

session.global_put(key, Oop::NIL)?;
```

Recipe 4: Commit a Write Safely

```
session.transaction(|session| {
    let value = session.new_string("committed")?;
    session.global_put("GemStoneRsCommitted", value)
})?;
```

The closure commits only when it returns `Ok`.

Recipe 5: Abort on Error

```
use gemstone_rs::Error;

let result: gemstone_rs::Result<()> = session.transaction(|session| {
    let value = session.new_string("will abort")?;
    session.global_put("GemStoneRsAborted", value)?;
    Err(Error::IllegalOop {
        operation: "intentional abort",
    })
});

assert!(result.is_err());
```

Recipe 6: Convert Rust Values to OOPs

```
use gemstone_rs::Value;

let seven = session.value_to_oop(&Value::SmallInt(7))?;
let yes = session.bool_oop(true);
let nothing = session.nil_oop();
```

Recipe 7: Fetch a String

```
let string_oop = session.new_string("hello")?;
let text = session.fetch_string(string_oop)?;
assert_eq!(text, "hello");
```

Recipe 8: Send a Message and Keep the Raw OOP

```
let seven = session.smallint_oop(7);
let printed_oop = session.perform_oop(seven, "printString", &[])?;
println!("{}", session.fetch_string(printed_oop)?);
```

Recipe 9: Send a Message and Marshal the Result

```
let seven = session.smallint_oop(7);
let value = session.perform(seven, "class", &[])?;
println!("{value:?}");
```

Recipe 10: Retain a Long-Lived OOP

```
let text = session.new_string("retained")?;
let handle = session.retain_oop(text)?;
println!("retained OOP {}", handle.oop().raw());
handle.release()?;
```

The handle releases automatically on drop if `release()` is not called.

Recipe 10A: Store a Rust Payload Under BridgeRoot

```
use gemstone_rs::{BridgeValue, Config, Session};

let mut session = Session::login(Config::from_env()?)?;
let payload = BridgeValue::dictionary([
    ("name".to_string(), BridgeValue::from("Tariq")),
    ("amount".to_string(), BridgeValue::from(100_i64)),
    ("currency".to_string(), BridgeValue::from("GBP")),
]);

let mut bridge_root = session.bridge_root()?;
```

```
bridge_root.put("MyTestDict", payload)?;
bridge_root.commit()?;
```

The default root is `UserGlobals at: #GemStoneRsBridgeRoot`.

Recipe 11: Browse Dictionaries

```
use gemstone_rs::browser::Browser;

let mut browser = Browser::new(&mut session);
for dictionary in browser.dictionaries()? {
    println!("{dictionary}");
}
```

Recipe 12: Browse a Class

```
use gemstone_rs::browser::{Browser, ALL_PROTOCOLS};

let mut browser = Browser::new(&mut session);
let protocols = browser.protocols("Object", false, "")?;
let methods = browser.methods("Object", ALL_PROTOCOLS, false, "")?;
let source = browser.source("Object", "printString", false, "")?;
```

Pass an empty dictionary to resolve through the active user's symbol list.

Recipe 13: Inspect an OOP From the CLI

```
gemstone-rs inspect oop 20
```

This prints the raw OOP, class OOP, and `printString`.

Recipe 14: Preview Codegen

```
gemstone-rs codegen preview examples/codegen/gemstone-rs.codegen
```

Use preview before writing generated files.

Recipe 15: Check Generated Code in CI

```
gemstone-rs codegen check examples/codegen/gemstone-rs.codegen
```

The repository runs the same check through `make verify`.

Recipe 16: Generate Wrappers

```
gemstone-rs codegen generate examples/codegen/gemstone-rs.codegen
```

Generated files are meant to be checked in.

Recipe 17: Discover a Config From a Live Stone

```
gemstone-rs codegen discover examples/codegen/discovered.codegen Object
```

Use this to bootstrap a config, then edit the generated mapping down to the API you actually want.

Recipe 18: Run the Local Explorer

```
gemstone-rs-explorer --port 8787
```

Open:

```
http://127.0.0.1:8787/
```

The explorer starts read-only and loopback-only.

Recipe 19: Enable Explorer Eval Explicitly

```
gemstone-rs-explorer --port 8787 --allow-eval
curl -s 'http://127.0.0.1:8787/api/eval?source=3%20%2B%204'
```

Use this locally only.

Recipe 20: Use the VS Code Workbench With a Checkout

```
{
  "gemstoneRs.checkoutPath": "/path/to/gemstone-rs",
  "gemstoneRs.useCargo": true,
  "gemstoneRs.codegenConfig": "examples/codegen/gemstone-rs.codegen"
}
```

Then open the GemStone RS sidebar and use the Dictionaries, Codegen Config, and Explorer trees.

Recipe 21: Verify Published Artifacts

```
scripts/publish_verify.sh 0.2.0
```

The script checks crates.io versions, installs the CLI and explorer, runs `--help`, and checks the Marketplace version.

gemstone-py vs gemstone-rs

`gemstone-py` and `gemstone-rs` solve related problems at different maturity levels.

Use `gemstone-py` when you want the most complete Python experience today: Python sessions, async examples, web framework adapters, native acceleration, codegen, VS Code workbench integration, and a Python database explorer.

Use `gemstone-rs` when you want Rust services, CLIs, workers, or tooling to talk to GemStone/S directly without keeping Python in the process.

Install Paths

Goal	gemstone-py	gemstone-rs
Application library	`python -m pip install gemstone-py`	`cargo add gemstone-rs`
Native acceleration	`python -m pip install "gemstone-py[fast]"`	Built into the Rust GCI path once `libgcirpc` is available

Goal	gemstone-py	gemstone-rs
Examples from source	`python -m pip install -e ".[examples]"`	`cargo run -p gemstone-rs --example quickstart`
CLI tooling	Python modules and examples	`cargo install gemstone-rs-cli`
Local explorer	`python-gemstone-database-explorer`	`cargo install gemstone-rs-explorer`
VS Code	`gemstone-py Workbench`	`gemstone-rs Workbench`

Both projects use the same GemStone-style environment:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=your-password
```

`GS_STONE_NAME` is accepted as a stone-name alias where the clients support it. Use `GS_LIB_PATH` when you want to point directly at a specific `libgcirpc` file.

Best Fit

Use case	Better choice	Why
Python web app	gemstone-py	FastAPI and Litestar examples are already first-class.
Python scripts and notebooks	gemstone-py	Lower friction for Python teams.
Rust service or worker	gemstone-rs	Native Rust API, Rust errors, Rust ownership, Cargo distribution.
Rust CLI tooling	gemstone-rs	CLI and explorer can run without Python.
VS Code database/ codegen workflow	gemstone-py today, gemstone-rs later	gemstone-py has the more mature explorer; gemstone-rs has the cleaner Rust backend direction.
Shared low-level bridge	gemstone-rs	The Rust API is the right place to isolate GCI safety and ownership.

Maturity Matrix

Capability	gemstone-py	gemstone-rs
Stable install path	Mature: PyPI and TestPyPI	Early: crates.io workflow and verification added
Native bridge	PyO3 extension path	Rust GCI crate and safe API
Sync API	Mature	Initial safe API
Async API	Mature enough for examples and tests	Not yet a core feature
Web frameworks	FastAPI, Litestar, Django examples	Future Axum/Actix examples recommended
Codegen	Python wrapper workflow	Rust wrapper workflow with preview/diff/check/generate
Browser API	Used by database explorer	CLI/explorer API for dictionaries/classes/methods/source
Local explorer	More mature Python app	Minimal Rust explorer proving the API
VS Code extension	More complete product flow	Thin command/workbench layer over Rust CLI and explorer
Live tests	Broader coverage	Growing smoke suite for login/eval/perform/globals/transactions/codegen
Release automation	More complete	Added crates/VSIX/GitHub verification path
Docs	Broader and polished	Catching up with guide, cookbook, article, comparison, PDFs

Feature Matrix

Feature	gemstone-py status	gemstone-rs status
Login/logout	Supported	Supported
`3 + 4 == 7` smoke	Supported	Supported

Feature	gemstone-py status	gemstone-rs status
Global put/get	Supported	Supported
String round-trip	Supported	Supported
`perform` and `perform_oop`	Supported	Supported
Commit/abort	Supported	Supported
OOP handles/export set	Supported	Supported
Browser dictionaries/classes/protocols/ methods/source	Supported	Supported through `Browser`, CLI, and explorer
Generated wrappers	Supported	Supported for selected classes/ methods
Codegen diff/check/generate	Supported	Supported
Live codegen discovery	Supported	Supported through CLI/API
FastAPI demo	Supported	Not applicable
Litestar demo	Supported	Not applicable
Axum/Actix demo	Not applicable	Planned
Database explorer	Mature Python app	Minimal Rust explorer
VS Code sidebar workflow	More complete	Initial Rust workbench

How They Should Work Together

The best long-term shape is not competition between the projects. It is a clean boundary:

- `gemstone-rs` owns the direct Rust/GCI interface and safe Rust API.
- `gemstone-py-native` can become a thin PyO3 layer over the Rust core.
- `gemstone-py` remains the Python API, examples, and Python web integration.
- The Python and Rust explorers can share concepts and eventually converge on common codegen workflows.

That gives Python users the friendliest path and Rust users a direct path, while keeping unsafe GCI behavior isolated in one Rust layer.

Recommendation

For production Python projects, start with `gemstone-py`.

For Rust-native services, CLIs, workers, and tooling, start with `gemstone-rs`, but treat it as an early API that should be validated against a live stone in your environment.

For VS Code and visual codegen work, use the existing Python explorer as the product reference and keep moving `gemstone-rs-explorer` toward the same level of capability.

BridgeRoot and Object Mapping

`gemstone-rs` now has a first object-mapping layer over the lower-level `Oop` API.

This is intentionally smaller than MagLev/GBS object mapping. It gives Rust code a bridge-root dictionary, typed Rust payload mapping, nested read-back, and generated mapping support while keeping direct OOP access available.

What Is Supported

The initial mapping layer supports:

- `Session::bridge_root()`
- `Session::bridge_root_named(name)`
- `BridgeRoot::put`
- `BridgeRoot::put_mapped`
- `BridgeRoot::get_oop`
- `BridgeRoot::get_value`
- `BridgeRoot::get_string`
- `BridgeRoot::get_smallint`
- `BridgeRoot::get_bool`
- `BridgeRoot::get_dictionary`
- `BridgeRoot::get_mapped`
- `BridgeRoot::remove`
- `BridgeRoot::commit`
- `BridgeRoot::commit_with_retry`
- `BridgeRoot::transaction`
- `BridgeDictionary::at_oop`
- `BridgeDictionary::at_value`
- `BridgeDictionary::at_string`
- `BridgeDictionary::at_smallint`
- `BridgeDictionary::at_bool`

- `BridgeDictionary::at_dictionary`
- `BridgeDictionary::at_mapped`
- `BridgeDictionary::at_vec`
- `BridgeKey`
- `BridgeKeyType::String`
- `BridgeKeyType::Symbol`
- `BridgeMapped`
- `#[derive(BridgeMapped)]`
- `BridgeFieldRead`
- `BridgeFieldWrite`
- `BridgeValue::Nil`
- `BridgeValue::Bool`
- `BridgeValue::SmallInt`
- `BridgeValue::String`
- `BridgeValue::Symbol`
- `BridgeValue::Oop`
- `BridgeValue::Dictionary`
- `BridgeValue::KeyedDictionary`
- `BridgeValue::Array`
- session-local OOP identity ids with `Session::identity_for_oop`

The default root is a GemStone `Dictionary` stored in `UserGlobals` under `#GemStoneRsBridgeRoot`.

Rust Example

```
use gemstone_rs::{BridgeValue, Config, Session};

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;

    let payload = BridgeValue::dictionary([
        ("name".to_string(), BridgeValue::from("Tariq")),
        ("amount".to_string(), BridgeValue::from(100_i64)),
        ("currency".to_string(), BridgeValue::from("GBP")),
    ]);

    let mut bridge_root = session.bridge_root()?;
    let payload_oop = bridge_root.put("MyTestDict", payload)?;
    let stored = bridge_root.get_oop("MyTestDict")?;
    assert_eq!(payload_oop, stored);

    bridge_root.commit()?;
    Ok(())
}
```

Run the example:

```
cargo run -p gemstone-rs --example bridge_root_mapping
```

Expected output includes:

```
bridge root: GemStoneRsBridgeRoot
MyTestDict OOP: <number>
```

Typed Rust Struct Mapping

For application code, implement `BridgeMapped` on a plain Rust type. The trait keeps the mapping explicit and reviewable while removing repeated dictionary field reads from application code.

```
use gemstone_rs::{BridgeDictionary, BridgeMapped, BridgeValue, Config, Session};

#[derive(Debug, Eq, PartialEq)]
struct BookingDraft {
    name: String,
    amount: i64,
    currency: String,
}

impl BridgeMapped for BookingDraft {
    fn to_bridge_value(&self) -> BridgeValue {
        BridgeValue::dictionary([
            ("name".to_string(), BridgeValue::from(self.name.clone())),
            ("amount".to_string(), BridgeValue::from(self.amount)),
            ("currency".to_string(), BridgeValue::from(self.currency.clone())),
        ])
    }

    fn from_bridge_dictionary(dictionary: &mut BridgeDictionary<'_>) ->
gemstone_rs::Result<Self> {
        Ok(Self {
            name: dictionary.at_string("name")?,
            amount: dictionary.at_smallint("amount")?,
            currency: dictionary.at_string("currency")?,
        })
    }
}

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;
    let mut bridge_root = session.bridge_root()?;

    let draft = BookingDraft {
        name: "Tariq".to_string(),
        amount: 100,
        currency: "GBP".to_string(),
    };
```

```
    bridge_root.put_mapped("MyTestDict", &draft)?;
    let loaded: BookingDraft = bridge_root.get_mapped("MyTestDict")?;
    assert_eq!(loaded, draft);

    bridge_root.commit()?;
    Ok(())
}
```

The checked-in example uses this pattern:

```
cargo run -p gemstone-rs --example bridge_root_mapping
```

Expected output includes:

```
bridge root: GemStoneRsBridgeRoot
MyTestDict OOP: <number>
loaded payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP" }
```

Derive-Based Mapping

For normal Rust structs, prefer `#[derive(BridgeMapped)]`. The derive writes a `BridgeMapped` implementation that stores fields in a GemStone dictionary and reads them back into Rust.

```
use gemstone_rs::{BridgeMapped, Config, Session};

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct CustomerDraft {
    #[bridge(key_type = "Symbol")]
    name: String,
}

#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    #[bridge(key = "amount", key_type = "Symbol")]
    amount: i64,
    customer: CustomerDraft,
    tags: Vec<String>,
}

fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;
    let mut bridge_root = session.bridge_root()?;

    let draft = BookingDraft {
        amount: 100,
        customer: CustomerDraft {
            name: "Tariq".to_string(),
```

```
        },
        tags: vec!["priority".to_string(), "demo".to_string()],
    };

    bridge_root.put_mapped("DerivedBookingDraft", &draft)?;
    let loaded: BookingDraft = bridge_root.get_mapped("DerivedBookingDraft")?;
    assert_eq!(loaded, draft);

    bridge_root.commit()?;
    Ok(())
}
```

Run the checked-in live example:

```
cargo run -p gemstone-rs --example derive_mapping
```

Expected output includes:

```
derived mapped payload: BookingDraft { amount: 100, customer: CustomerDraft { name:
"Tariq" }, tags: ["priority", "demo"] }
bridge root identity: <number>
```

Nested Read-Back

Nested mapped structs and arrays round-trip through the same bridge dictionary format:

Rust field	GemStone storage	Read-back helper
`String`	`String`	`BridgeFieldRead` / `at_string`
`i64`	`SmallInteger`	`BridgeFieldRead` / `at_smallint`
`bool`	`true` / `false`	`BridgeFieldRead` / `at_bool`
`Oop`	raw object reference	`BridgeFieldRead` / `at_oop`
another `BridgeMapped` struct	nested `Dictionary`	`BridgeDictionary::at_mapped`
`Vec<T>`	`Array`	`BridgeDictionary::at_vec`

This lets a Rust payload keep normal nested shape:

```
#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    customer: CustomerDraft,
```

```
    tags: Vec<String>,
}
```

When read-back fails, mapping errors include field context:

```
field amount expected GemStone value type SmallInt, got Oop 1234
```

Key Policy

GemStone dictionaries often use either string keys or symbol keys. The mapping layer makes that explicit:

```
#[derive(Clone, Debug, Eq, PartialEq, BridgeMapped)]
struct BookingDraft {
    #[bridge(key = "amount", key_type = "Symbol")]
    amount: i64,
}
```

Manual code can use the same policy:

```
use gemstone_rs::BridgeKeyType;

bridge_root.put_with_key_type("BookingDraft", BridgeKeyType::Symbol, "stored")?;
let oop = bridge_root.get_oop_with_key_type("BookingDraft", BridgeKeyType::Symbol)?;
```

Use string keys when interoperating with JSON-like payloads and symbol keys when you want Smalltalk-style dictionary access.

Transactions and Identity

`BridgeRoot::transaction` commits on success and aborts on error:

```
bridge_root.transaction(|root| {
    root.put_mapped("BookingDraft", &draft)?;
    let loaded: BookingDraft = root.get_mapped("BookingDraft")?;
    assert_eq!(loaded, draft);
    Ok(())
})?;
```

For conflict-prone commits, use a bounded retry:

```
bridge_root.put_mapped("BookingDraft", &draft)?;
bridge_root.commit_with_retry(2)?;
```

`Session` also tracks a session-local identity id for OOPs it sees:

```
let oop = bridge_root.get_oop("BookingDraft")?;
let identity = bridge_root.identity_id();
println!("bridge root identity={identity}, payload oop={}", oop.raw());
```

That is not transparent object persistence. It is a stable in-session cache key for wrappers, inspectors, and explorer views.

Comparison With MagLev/GBS

The MagLev branch example uses:

```
session bridgeRoot at: #MyTestDict put: payload.
session commitTransactionOrSignalConflict
```

The closest current `gemstone-rs` shape is:

```
let mut bridge_root = session.bridge_root()?;
bridge_root.put("MyTestDict", payload)?;
bridge_root.commit()?;
```

For typed Rust structs:

```
bridge_root.put_mapped("MyTestDict", &draft)?;
let loaded: BookingDraft = bridge_root.get_mapped("MyTestDict")?;
```

This is still explicit mapping, not transparent persistence. The benefit is that Rust teams can review every field mapping and still keep direct OOP access available when they need it.

Codegen Support

`gemstone-rs` codegen can emit `BridgeMapped` structs from config:

```
mapped = BookingDraft | doc=A typed Rust payload stored under BridgeRoot.
field = BookingDraft.name | type=String | key=name
field = BookingDraft.amount | type=SmallInt | key=amount | key_type=Symbol
```

```
field = BookingDraft.currency | type=String | key=currency
field = BookingDraft.tags | type=Vec<String> | key=tags
```

Generate a mapping config proposal from a live GemStone class:

```
gemstone-rs codegen discover-mapping gemstone-rs.codegen BookingDraft Object
```

Run the live discovery example:

```
cargo run -p gemstone-rs --example codegen_discover_mapping
```

Run the generated mapping example:

```
cargo run -p gemstone-rs --example generated_mapping_app
```

This is the recommended path for repeated mappings because the generated source is checked in and reviewed like any other Rust code.

Explorer and VS Code Support

The local explorer exposes BridgeRoot-oriented endpoints:

```
curl -s http://127.0.0.1:8787/api/bridge/root
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/mapping-config?mapped=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/codegen/discover-mapping?
mapped=BookingDraft&class=Object'
```

The VS Code workbench adds the same object-mapping workflow:

- `GemStone RS: Generate Mapping Config`
- `GemStone RS: Preview BridgeRoot`
- `GemStone RS: Run Generated Mapping Example`

The sidebar also shows these actions under `Codegen Config`.

Current Boundaries

The mapping layer is explicit, not transparent persistence. It does not yet make every GemStone object look like a native Rust object automatically. The current design is deliberately conservative:

- `Oop` remains visible and available.

- `Session` is still non-`Send` and non-`Sync`.
- writes happen only through explicit `put`, `put_mapped`, or generated code.
- the identity map is session-local and does not replace GemStone identity.

Rust Codegen

`gemstone-rs` codegen creates small Rust wrapper structs around GemStone OOPs. The goal is to move repeated selector strings into checked-in generated code while keeping the mapping reviewable.

Install

```
cargo install gemstone-rs-cli
gemstone-rs codegen --help
```

From a checkout:

```
cargo run -p gemstone-rs-cli -- codegen preview examples/codegen/gemstone-rs.codegen
```

Config Format

The config is intentionally line-oriented:

```
output = examples/codegen/generated/gemstone_wrappers.rs
class = Object
method = Object>>printString | return=String | doc=Return the receiver printString.
method = Object>>class
```

Use `Dictionary:ClassName` when a class should resolve from a specific dictionary:

```
class = UserGlobals:OkzBooking
method = UserGlobals:OkzBooking>>findById: | args=id | return=Oop | doc=Find a
booking by id.
```

Class-side references use `class`:

```
class = UserGlobals:OkzBooking class
method = UserGlobals:OkzBooking class>>findById: | args=id | return=Oop
```

Method Metadata

Option	Example	Purpose
`args`	`args=id,user`	Names generated Rust function arguments.
`return`	`return=String`	Generates a typed return helper instead of `Value`.
`doc`	`doc=Find by id.`	Writes a Rust doc comment above the generated method.

Supported return types:

Config value	Rust return
`Value`	`gemstone_rs::Value`
`String`	`String`
`SmallInt`	`i64`
`Bool`	`bool`
`Oop`	`gemstone_rs::Oop`

Mapped Structs

Codegen can also emit typed `BridgeMapped` structs for values stored under `BridgeRoot`:

```
mapped = BookingDraft | doc=A typed Rust payload stored under BridgeRoot.
field = BookingDraft.name | type=String | key=name
field = BookingDraft.amount | type=SmallInt | key=amount | key_type=Symbol
field = BookingDraft.currency | type=String | key=currency
field = BookingDraft.tags | type=Vec<String> | key=tags
```

Supported field types are `String`, `SmallInt`, `Bool`, `Oop`, `Mapped<OtherStruct>`, and `Vec<T>`.

Field options:

Option	Example	Purpose
`type`	`type=Vec<String>`	Rust/GemStone field conversion.
`key`	`key=amount`	GemStone dictionary key.
`key_type`	`key_type=Symbol`	Use string keys or symbol keys explicitly.
`doc`	`doc=Booking amount.`	Writes a Rust doc comment above the field.

Commands

Create a starter config:

```
gemstone-rs codegen init gemstone-rs.codegen
```

Generate a config from a live stone:

```
gemstone-rs codegen discover gemstone-rs.codegen Object
gemstone-rs codegen discover gemstone-rs.codegen UserGlobals:OkzBooking
gemstone-rs codegen discover-mapping gemstone-rs.codegen BookingDraft Object
```

From a checkout:

```
cargo run -p gemstone-rs --example codegen_discover
cargo run -p gemstone-rs --example codegen_discover_mapping
```

Preview without writing:

```
gemstone-rs codegen preview gemstone-rs.codegen
```

Show a generated diff:

```
gemstone-rs codegen diff gemstone-rs.codegen
```

Check freshness in CI:

```
gemstone-rs codegen check gemstone-rs.codegen
```

Write generated wrappers:

```
gemstone-rs codegen generate gemstone-rs.codegen
```

Generated Shape

For:

```
method = Object>>printString | return=String | doc=Return the receiver printString.
```

The generated wrapper includes:

```
pub fn print_string(&mut self) -> Result<String> {
    let value = self.session.perform(self.oop, "printString", &[])?;
    match value {
        Value::String(value) => Ok(value),
        Value::Oop(oop) => self.session.fetch_string(oop),
        other => Err(Error::UnexpectedType {
            expected: "String",
            actual: format!("{other:?}"),
        }),
    }
}
```

Generated files are meant to be checked in. That makes diffs, reviews, and editor indexing predictable.

Generated Wrapper App

The repository includes a small live example that imports the checked-in generated wrapper and calls `Object>>printString` on a small integer:

```
cargo run -p gemstone-rs --example generated_wrapper_app
```

Expected output:

```
generated wrapper printString: 7
```

The example uses the generated `Object::from_oop` constructor:

```
#[path = "../../../examples/codegen/generated/gemstone_wrappers.rs"]
mod gemstone_wrappers;

use gemstone_rs::{Config, Session};
```

```
fn main() -> gemstone_rs::Result<()> {
    let mut session = Session::login(Config::from_env()?)?;
    let oop = session.smallint_oop(7);
    let mut object = gemstone_wrappers::Object::from_oop(&mut session, oop);
    println!("{}", object.print_string()?);
    Ok(())
}
```

Generated Object Mapping

The generated file can also include Rust data structs that implement `BridgeMapped`. Run:

```
cargo run -p gemstone-rs --example generated_mapping_app
```

Expected output:

```
generated mapped payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP",
tags: ["priority", "demo"] }
```

The config-driven mapping emits a struct like:

```
#[derive(Clone, Debug, Eq, PartialEq)]
pub struct BookingDraft {
    pub name: String,
    pub amount: i64,
    pub currency: String,
    pub tags: Vec<String>,
}
```

and an implementation of `BridgeMapped` so it works with:

```
bridge_root.put_mapped("GeneratedBookingDraft", &draft)?;
let loaded: BookingDraft = bridge_root.get_mapped("GeneratedBookingDraft")?;
```

VS Code Workflow

The `gemstone-rs Workbench` extension exposes the same flow in the GemStone RS sidebar:

- Discover from Live Stone
- Generate Mapping Config
- Preview BridgeRoot
- Run Generated Mapping Example

- Preview Wrappers
- Diff Generated Output
- Check Freshness
- Generate Wrappers
- Open Codegen Docs

`Generate Wrappers` shows the diff first and asks before writing when output would change.

Explorer Guide

`gemstone-rs-explorer` is a local-only HTTP explorer built on the Rust API. It is useful for proving the browser, inspect, eval, and codegen surfaces before embedding richer tooling in VS Code or another frontend.

Install and Run

```
cargo install gemstone-rs-explorer
gemstone-rs-explorer --port 8787
```

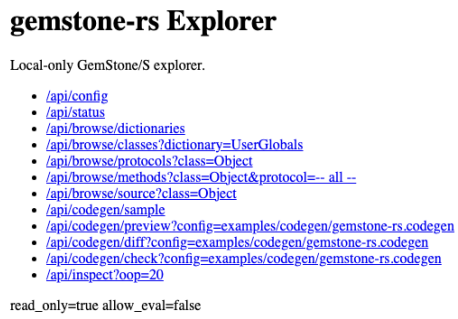
From a checkout:

```
cargo run -p gemstone-rs-explorer -- --port 8787
```

Open:

```
http://127.0.0.1:8787/
```

Screenshot:



gemstone-rs Explorer

Local-only GemStone/S explorer.

- /api/config
- /api/status
- /api/browse/dictionaries
- /api/browse/classes?dictionary=UserGlobals
- /api/browse/protocols?class=Object
- /api/browse/methods?class=Object&protocol=-- all --
- /api/browse/source?class=Object
- /api/codegen/sample
- /api/codegen/preview?config=examples/codegen/gemstone-rs.codegen
- /api/codegen/diff?config=examples/codegen/gemstone-rs.codegen
- /api/codegen/check?config=examples/codegen/gemstone-rs.codegen
- /api/inspect?oop=20

read_only=true allow_eval=false

Safety Defaults

The explorer:

- binds to `127.0.0.1` by default
- rejects non-loopback hosts
- starts read-only
- requires `--allow-eval` before workspace evaluation
- requires `--allow-write` before codegen writes
- reads GemStone credentials from the same `GS_*` environment as the CLI

Do not expose it publicly without adding authentication and transport security.

Browse Endpoints

Run these from a second terminal:

```
curl -s http://127.0.0.1:8787/health
curl -s http://127.0.0.1:8787/api/config
curl -s http://127.0.0.1:8787/api/status
curl -s http://127.0.0.1:8787/api/browse/dictionaries
```

```
curl -s 'http://127.0.0.1:8787/api/browse/classes?dictionary=UserGlobals'
curl -s 'http://127.0.0.1:8787/api/browse/protocols?class=Object&meta=0'
curl -s 'http://127.0.0.1:8787/api/browse/methods?class=Object&protocol=--%20all%20--
&meta=0'
curl -s 'http://127.0.0.1:8787/api/browse/source?
class=Object&selector=printString&meta=0'
curl -s 'http://127.0.0.1:8787/api/inspect?oop=20'
```

Expected shapes:

```
{"status":"ok"}
{"host":"127.0.0.1","port":8787,"readOnly":true,"allowEval":false}
{"connected":true,"sessionId":12345,"needsCommit":false,"inTransaction":false}
{"success":true,"dictionaries":["UserGlobals"]}
```

Eval

Eval is disabled by default:

```
gemstone-rs-explorer --allow-eval
```

Then:

```
curl -s 'http://127.0.0.1:8787/api/eval?source=3%20%2B%204'
```

Expected:

```
{"type":"smallInt","value":7}
```

Codegen Endpoints

Read-only endpoints:

```
curl -s http://127.0.0.1:8787/api/codegen/sample
curl -s 'http://127.0.0.1:8787/api/codegen/preview?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/diff?config=examples/codegen/gemstone-
rs.codegen'
curl -s 'http://127.0.0.1:8787/api/codegen/check?config=examples/codegen/gemstone-
rs.codegen'
```

```
curl -s 'http://127.0.0.1:8787/api/codegen/discover-mapping?
mapped=BookingDraft&class=Object'
```

Expected check result when generated output is current:

```
{"success":true,"exists":true,"upToDate":true}
```

Write-gated endpoint:

```
gemstone-rs-explorer --allow-write
```

```
curl -s 'http://127.0.0.1:8787/api/codegen/generate?config=examples/codegen/gemstone-
rs.codegen'
```

Expected:

```
{"success":true,"output":"examples/codegen/generated/
gemstone_wrappers.rs","bytes":1234}
```

BridgeRoot Endpoints

BridgeRoot inspection and mapping-config preview use the same live session configuration as the browser endpoints:

```
curl -s http://127.0.0.1:8787/api/bridge/root
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft'
curl -s 'http://127.0.0.1:8787/api/bridge/get?key=BookingDraft&key_type=Symbol'
curl -s 'http://127.0.0.1:8787/api/bridge/mapping-config?mapped=BookingDraft'
```

Expected shapes:

```
{"success":true,"name":"GemStoneRsBridgeRoot","oop":1234,"identityId":1}
{"oop":5678,"classOop":9012,"printString":"aDictionary(...)"}
{"success":true,"config":"mapped = BookingDraft ..."}
```

Product Direction

The explorer should remain a separate binary crate. The core `gemstone-rs` library stays focused on safe GemStone API access, while the explorer proves higher-level browsing and codegen workflows.

Good next steps:

- richer frontend over the current HTTP endpoints
- generated wrapper preview and file diff UI
- explicit codegen selection config
- optional VS Code webview embedding

VS Code Workbench

`gemstone-rs Workbench` is the VS Code companion for the Rust CLI and local explorer. It keeps the CLI as the stable contract and adds a sidebar, output panel, generated-file previews, and codegen diff prompts.

Marketplace:

```
https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-rs-workbench
```

Setup

For installed CLI tools:

```
{
  "gemstoneRs.cliPath": "gemstone-rs",
  "gemstoneRs.explorerPath": "gemstone-rs-explorer",
  "gemstoneRs.codegenConfig": "gemstone-rs.codegen"
}
```

For a source checkout:

```
{
  "gemstoneRs.checkoutPath": "/path/to/gemstone-rs",
  "gemstoneRs.useCargo": true,
  "gemstoneRs.codegenConfig": "examples/codegen/gemstone-rs.codegen"
}
```

Use the same `GS_*` environment as the CLI.

Sidebar Tree

Open the GemStone RS activity bar item.

The sidebar contains:

- `Dictionaries`
- `Codegen Config`
- `Explorer`

`Dictionaries` expands live dictionaries, classes, protocols, and methods. Selecting a method opens its source in an untitled Smalltalk editor.

`Codegen Config` exposes:

- Discover from Live Stone
- Generate Mapping Config
- Preview BridgeRoot
- Run Generated Mapping Example
- Preview Wrappers
- Diff Generated Output
- Check Freshness
- Generate Wrappers
- Open Codegen Docs

`Explorer` exposes:

- Verify Setup
- Eval Smalltalk
- Launch Explorer

Command Palette

Commands:

- `GemStone RS: Verify Setup`
- `GemStone RS: Eval Smalltalk`
- `GemStone RS: Browse Dictionaries`
- `GemStone RS: Browse Classes`
- `GemStone RS: Codegen Init`
- `GemStone RS: Codegen Discover`
- `GemStone RS: Generate Mapping Config`
- `GemStone RS: Preview BridgeRoot`
- `GemStone RS: Run Generated Mapping Example`
- `GemStone RS: Codegen Preview`
- `GemStone RS: Codegen Diff`
- `GemStone RS: Codegen Check`
- `GemStone RS: Codegen Generate`
- `GemStone RS: Launch Explorer`
- `GemStone RS: Open Method Source`

- `GemStone RS: Open Codegen Docs`

Codegen Workflow

1. Set `gemstoneRs.codegenConfig`.
2. Run `GemStone RS: Codegen Discover` or edit the config manually.
3. Run `GemStone RS: Codegen Preview`.
4. Run `GemStone RS: Codegen Diff`.
5. Run `GemStone RS: Codegen Generate`.

`Codegen Generate` runs the diff first. If output would change, it opens the diff and asks before writing.

Object Mapping Workflow

Use `GemStone RS: Generate Mapping Config` when you want the live stone to inspect a GemStone class and propose a `BridgeMapped` config. The command asks for:

- config path
- Rust struct name
- GemStone class reference, such as `Object` or `UserGlobals:OkzBooking`

Use `GemStone RS: Preview BridgeRoot` after starting the local explorer. It opens:

```
http://127.0.0.1:8787/api/bridge/root
```

Use `GemStone RS: Run Generated Mapping Example` to run:

```
cargo run -p gemstone-rs --example generated_mapping_app
```

Develop the Extension

```
cd vscode-gemstone-rs-workbench
npm ci
npm run check
npm run package -- --out gemstone-rs-workbench-0.2.0.vsix
```

From the repository root:

```
make vscode-package
```

Later Feature Work

Embedding `gemstone-rs-explorer` as a VS Code webview should remain a feature release because it changes the user experience and test surface. The current extension intentionally launches the explorer externally and opens the local URL.

Release Checklist

Use this checklist for coordinated crate, VSIX, and GitHub releases.

Before Release

- Update crate versions in `Cargo.toml` files.
- Update `vscode-gemstone-rs-workbench/package.json`.
- Regenerate codegen examples:

```
cargo run -p gemstone-rs-cli -- codegen generate examples/codegen/gemstone-rs.codegen
```

- Run local verification:

```
make verify
make vscode-package
python3 docs/build_pdf_docs.py
```

`make verify` checks that PDF generation completes and produces non-empty PDF files. The release workflow rebuilds and attaches fresh PDFs for the target runner because WeasyPrint output can differ byte-for-byte across platforms.

Publish

Set GitHub secrets once:

```
gh secret set CARGO_REGISTRY_TOKEN
gh secret set VSCE_PAT
```

Run the release workflow:

```
gh workflow run release.yml \
  --ref main \
```

```
  -f version=0.2.0 \
  -f publish-crates=true \
  -f publish-vsix=true \
  -f create-github-release=true \
  -f dry-run=false
```

The workflow publishes crates in dependency order:

1. `gemstone-gci`
2. `gemstone-rs-macros`
3. `gemstone-rs`
4. `gemstone-rs-cli`
5. `gemstone-rs-explorer`

It also packages/publishes the VSIX and can create the GitHub release with the VSIX attached.

Manual crate publishing remains available:

```
scripts/publish_crates.sh
```

For a dry run:

```
DRY_RUN=1 scripts/publish_crates.sh
```

Dry-run mode checks each crate's publish file list locally. The real publish path still uses `cargo publish` in dependency order so crates.io resolves each new dependency after it has been published. If a workflow is rerun after a partial publish, already-published crate versions are skipped and the remaining crates continue.

Post-Release Verification

```
scripts/publish_verify.sh 0.2.0
```

The script checks:

- crates.io package/version JSON for all crates
- `cargo install gemstone-rs-cli`
- `cargo install gemstone-rs-explorer`
- `gemstone-rs --help`
- `gemstone-rs-explorer --help`
- VS Code Marketplace version matches `package.json`
- GitHub Release `v<version>` exists
- GitHub Release assets include the VSIX and `SHA256SUMS`

To skip the GitHub Release asset check during early testing:

```
VERIFY_GITHUB_RELEASE=0 scripts/publish_verify.sh 0.2.0
```

Optional Live Smoke

Run the manual GitHub Actions live job with GemStone secrets configured, or run locally:

```
GS_RUN_LIVE_RUST=1 cargo test -p gemstone-rs live_ -- --test-threads=1
```

The live smoke coverage includes login/logout, `3 + 4 == 7`, global put/get, string round-trip, `perform`, commit, abort, browser lookup of `Object`, and generated wrapper `printString`.

The `--test-threads=1` flag is intentional. GemStone GCI has process-global session state, so live tests should not be run concurrently.

> This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.